

VCGen

SeaHorn Style

Idea

SeaHorn-style VCGen keeps control flow explicit.

For each block i , introduce a Boolean reachability variable:

$$bb_i$$

Then constrain the selected blocks so they describe an entry-to-exit path.

Formula Shape

$$\text{VCGen_SeaHorn}(P) = \text{CFG} \wedge \text{DEF} \wedge \text{JUMPS} \wedge \text{PHI} \wedge \text{ERROR}$$

- CFG: selected block variables form a path.
- DEF: assignments hold when their block is selected.
- JUMPS: selected branch edges satisfy branch conditions.
- PHI: selected incoming edges choose phi values.
- ERROR: the selected exit violates the assertion.

CFG Constraints

$$\begin{aligned} \text{CFG} &= \text{bb}_{\text{entry}} \\ &\wedge \forall i \neq \text{entry}. \text{bb}_i \Rightarrow \bigvee_{j \in \text{pred}(i)} \text{bb}_j \\ &\wedge \text{bb}_{\text{exit}} \end{aligned}$$

Concrete encoders may add edge variables, forward constraints, dominator constraints, or at-most-one constraints. The semantic role is the same: describe feasible control.

Running Diamond

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join

right:
  a_right := y + 1
  goto join
```

Join And Error

```
join:
  a := phi [left: a_left, right: a_right]
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

Encoded Pieces

$$\begin{aligned}\mathbf{DEF} = & (\mathbf{bb}_{\text{entry}} \Rightarrow c = (x < y)) \\ & \wedge (\mathbf{bb}_{\text{left}} \Rightarrow a_{\text{left}} = x + 1) \\ & \wedge (\mathbf{bb}_{\text{right}} \Rightarrow a_{\text{right}} = y + 1) \\ & \wedge (\mathbf{bb}_{\text{join}} \Rightarrow \text{ok} = (a > 0))\end{aligned}$$

$$\begin{aligned}\mathbf{JUMPS} = & ((\mathbf{bb}_{\text{entry}} \wedge \mathbf{bb}_{\text{left}}) \Rightarrow c) \\ & \wedge ((\mathbf{bb}_{\text{entry}} \wedge \mathbf{bb}_{\text{right}}) \Rightarrow \neg c)\end{aligned}$$

$$\begin{aligned}\mathbf{PHI} = & ((\mathbf{bb}_{\text{left}} \wedge \mathbf{bb}_{\text{join}}) \Rightarrow a = a_{\text{left}}) \\ & \wedge ((\mathbf{bb}_{\text{right}} \wedge \mathbf{bb}_{\text{join}}) \Rightarrow a = a_{\text{right}})\end{aligned}$$

Error Condition

For the SASA exit block:

```
exit:  
  assume pre  
  assert post  
  halt
```

emit:

$$\mathbf{ERROR} = \text{bb}_{\text{exit}} \Rightarrow \text{pre} \wedge \neg \text{post}$$

Together with `bb_exit`, this asks for a complete failing execution.

Optimization: Unguarded DEF

Static definitions may be exposed as top-level equations:

Guarded:

$$\text{bb}_i \Rightarrow x = y + 1$$

Unguarded:

$$x = y + 1$$

The unguarded form helps substitution and algebraic simplification, but only when the definition and its side conditions are globally safe.

Optimization: Phi As ITE

Edge implications:

$$(bb_i \wedge bb_j) \Rightarrow x = x_i$$

$$(bb_k \wedge bb_j) \Rightarrow x = x_k$$

One defining equation:

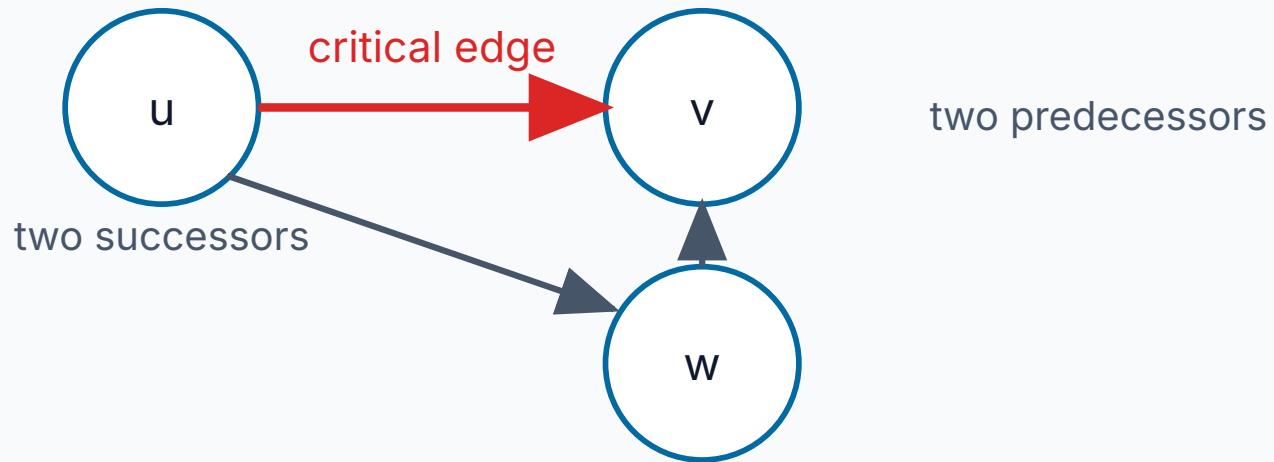
$$x = \text{ite}(bb_i, x_i, x_k)$$

After Boolean propagation learns bb_i , the ITE collapses to $x = x_i$.

Pitfall: Critical Edges

An edge $u \rightarrow v$ is critical when:

$$|\text{succ}(u)| > 1 \wedge |\text{pred}(v)| > 1$$



Block variables alone do not identify which incoming edge was taken.

Critical Edge Example

```
entry:
  c := havoc
  if c goto join else mid

mid:
  goto join

join:
  ok := phi [entry: true, mid: false]
```

Bad true-edge encoding:

$$(bb_{\text{entry}} \wedge bb_{\text{join}}) \Rightarrow c$$

On path entry -> mid -> join, both blocks are selected while c is false. The real bug path can be incorrectly rejected.

Pitfall: ITE Merge Exclusivity

If both predecessors can be selected, independent ITE merges can create a mixed state:

$$\begin{aligned}x &= \text{ite}(\text{bb}_{\text{left}}, x_{\text{left}}, x_{\text{right}}) \\p &= \text{ite}(\text{bb}_{\text{right}}, p_{\text{right}}, p_{\text{left}})\end{aligned}$$

Repair:

$$\neg(\text{bb}_{\text{left}} \wedge \text{bb}_{\text{right}})$$

At most one incoming predecessor may feed a merge.